Real-Time Video Generation With MiniMax H3

8× RTX PRO 5000 · vLLM-Omni

Shunyang Li | Public technical brief | October 2026



Agenda

01 Toward Real-Time H3 Generation
02 vLLM-Omni and ComfyUI Applications

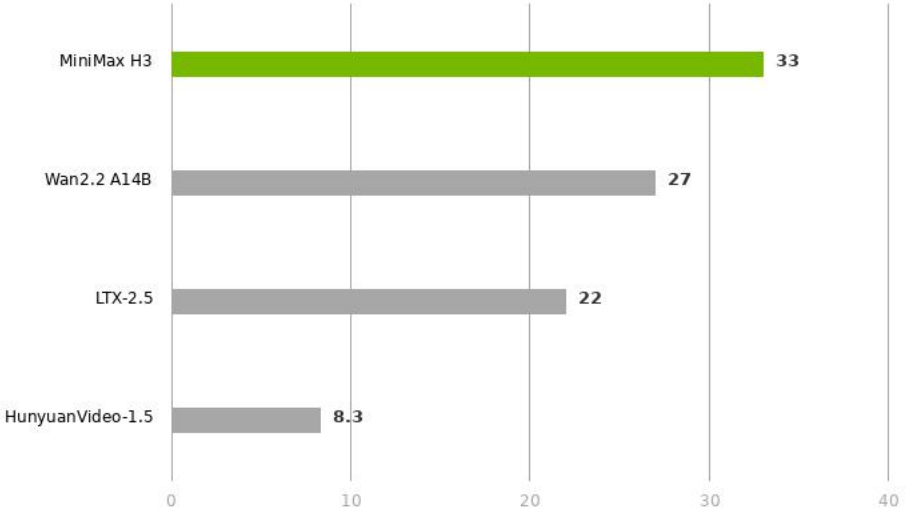


01 · Real-Time H3 Generation



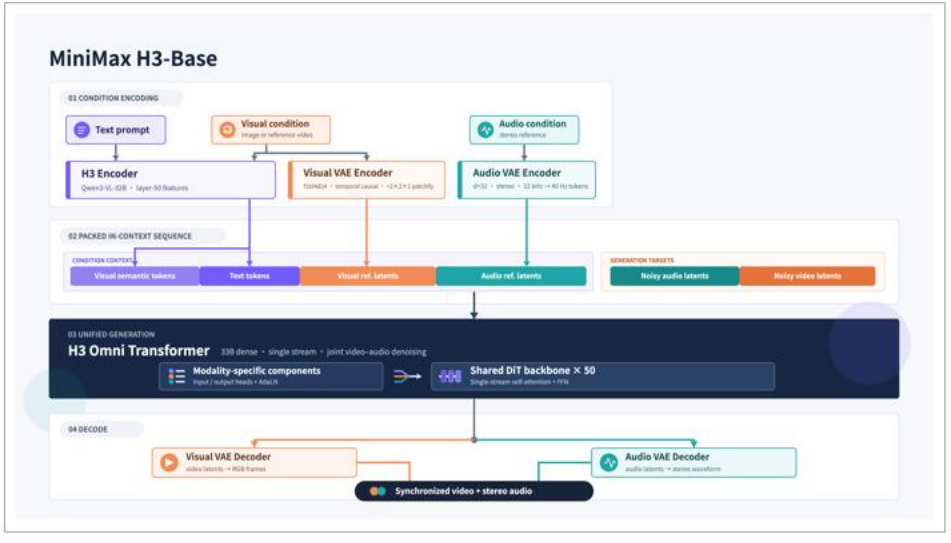
Real-Time Ref2VA Raises the Systems Bar

A 124-frame clip at 24 FPS is real time only when end-to-end latency stays below 5.167 s (RTF < 1)



Generative Backbone · Full Parameters (B)

RTF = latency ÷ media duration; the 124-frame deadline is 5.167 s. H3 has the largest dense backbone in this peer set; Wan2.2 has 27B total and 14B active parameters



Model Capability · Native 768P · 33B Dense

32B encoder → 33B dense Omni Transformer → visual/audio VAE. Fifty blocks × 50 steps keep compute and memory pressure high; about 13B AdaLN parameters can be cached

Sources: MiniMax H3, Wan2.2, LTX-2.5 and HunyuanVideo-1.5 official releases · author 960×544 Ref2VA workload

SM120 Brings FP4 Compute; PCIe Limits Scale

All discrete SM120 cards pair fifth-generation Tensor Cores with GDDR7 and PCIe Gen5; none provides GPU-to-GPU NVLink

SKU	Compute	Memory	GPU I/O / Fabric
RTX PRO 5000 ×8 Blackwell	SM120 · 110 SMs 5th-gen Tensor · FP4	72 GB GDDR7 ECC 48 GB variant	PCIe Gen5 ×16 Planned: NIC → RDMA*
RTX PRO 6000 WS / Server	SM120 · 188 SMs 5th-gen Tensor · FP4	96 GB GDDR7 ECC	PCIe Gen5 ×16 No GPU NVLink
GeForce RTX 5090	SM120 · 170 SMs 5th-gen Tensor · FP4	32 GB GDDR7	PCIe Gen5 ×16 No NVLink
DGX Spark · GB10	SM121 · 48 SMs 5th-gen Tensor · FP4	128 GB unified LPDDR5X	NVLink-C2C in SoC 200 Gb/s ConnectX-7

* Planned system path; GPUDirect RDMA requires a compatible NIC and PCIe topology · Sources: NVIDIA product specs, CUDA GPU table, GPUDirect RDMA guide

Latency Breakdown

NCCL occupies 49.2% of the average GPU timeline

Stage	Average GPU Time*	Share
Mean GPU timeline*	250.0 s	100.0%
NCCL communication	123.0 s	49.2%
↳ DiT collectives	116.1 s	46.4%
↳ Encoder, VAE, and other collectives	7.0 s	2.8%
Tensor compute	84.5 s	33.8%
↳ Attention	45.3 s	18.1%
↳ GEMM / MLP	39.2 s	15.7%
AdaLN weight staging + device layout	38.5 s	15.4%
Encoder, VAE, and other GPU work	4.0 s	1.6%

- Of the 38.5 s average, 36.9 s is AdaLN projection weight staging from CPU; layout is 1.5 s
- This run uses the default runtime cache, not a complete precomputed schedule sidecar

Base Ref2VA · 960×544 · 124f @ 24 FPS · 50 steps · BF16 · 8× RTX PRO 5000 · default AdaLN offload · local NCCL / RDMA off · *sum across 8 GPU timelines ÷ 8

How Denoising Works

H3 follows a learned velocity field from noise toward clean video and audio latents

State and direction

- x(t) is the current latent; the DiT predicts a data-ward velocity

Solver update

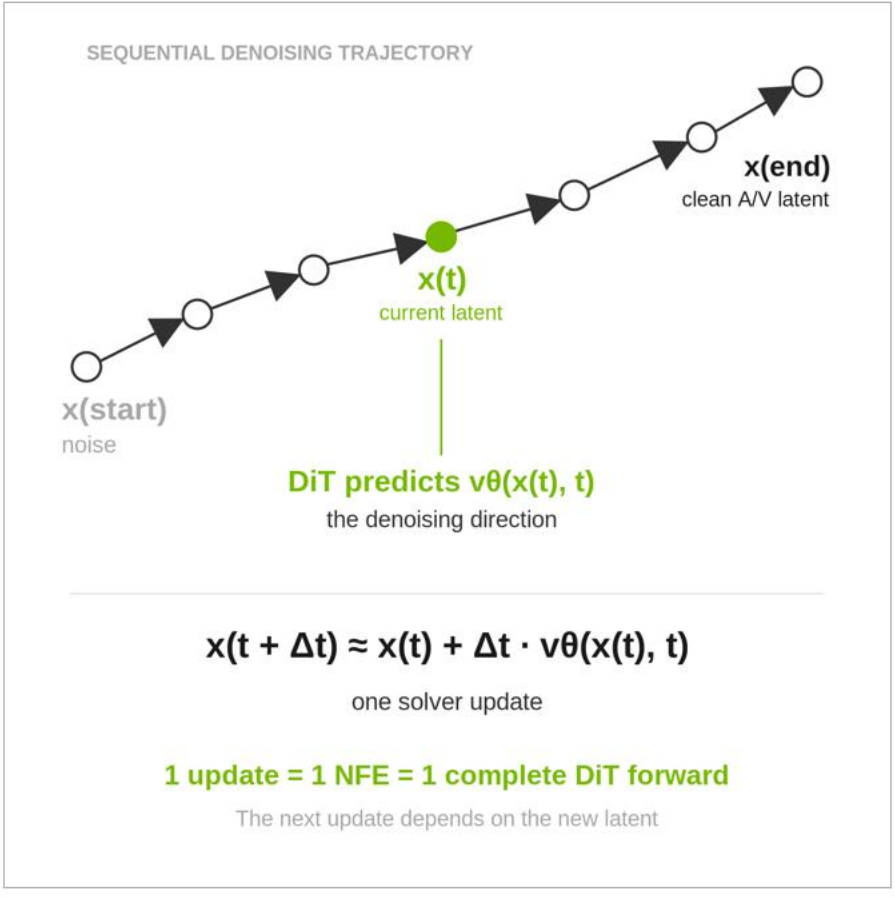
- Euler advances x(t) by a small interval along that velocity
- The next DiT call needs the new latent, so conventional steps run in sequence

Network function evaluation (NFE)

- One NFE is one complete DiT forward

Base H3

- 50 NFE × 50 main blocks = 2,500 main-block passes



Classical Flow-Matching Sampling

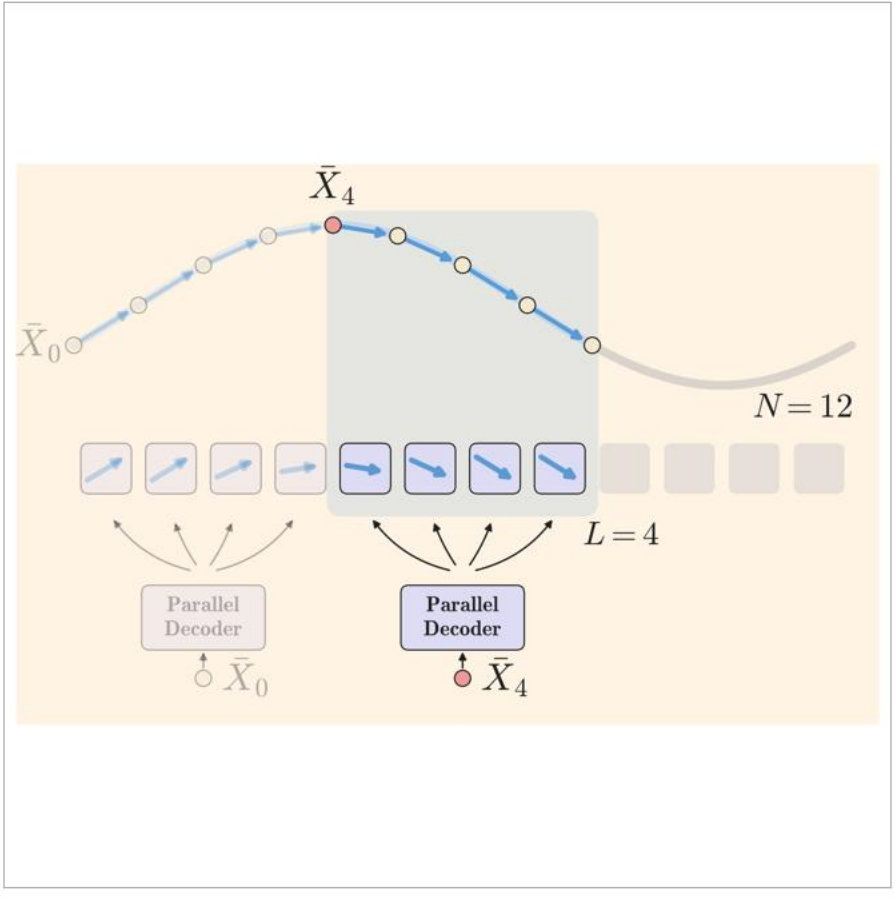
A chain of dependent solver updates turns noise into clean latents

Sources: MiniMax H3 Base configuration · NVIDIA FastGen-PDD sampling background (arXiv:2607.26004) · author abstraction

Few-Step Distillation

PDD predicts four denoising intervals per DiT call: 50 calls → 8

- Base H3 calls the full DiT 50 times. Each call predicts one short interval, updates the latent, and provides the starting point for the next call
- PDD uses Base H3 as the teacher and trains a student on groups of four adjacent denoising intervals
- One student call predicts four consecutive directions from the same noisy latent
- The solver combines those directions, advances the latent, and invokes the student again
- The released H3 schedule covers 32 fine intervals in eight student calls
- **50 → 8 DiT calls; 2,500 → 400 repeated block passes (−84%)**



N = fine intervals · L = intervals predicted in parallel

Paper example: N = 12, L = 4 · H3 PDD8: N = 32, L = 4

Sources: NVIDIA FastGen-PDD, Fig. 2 (arXiv:2607.26004) · Alibaba-PAI MiniMax-H3 Ref2VA Acc-8Step @ 335001f

Sparse Attention

VSA scans globally, then spends fine attention only where it matters

Why dense attention hurts

- 34,373 tokens create 1.181B candidate pairs per head and block

Look broadly, then zoom in

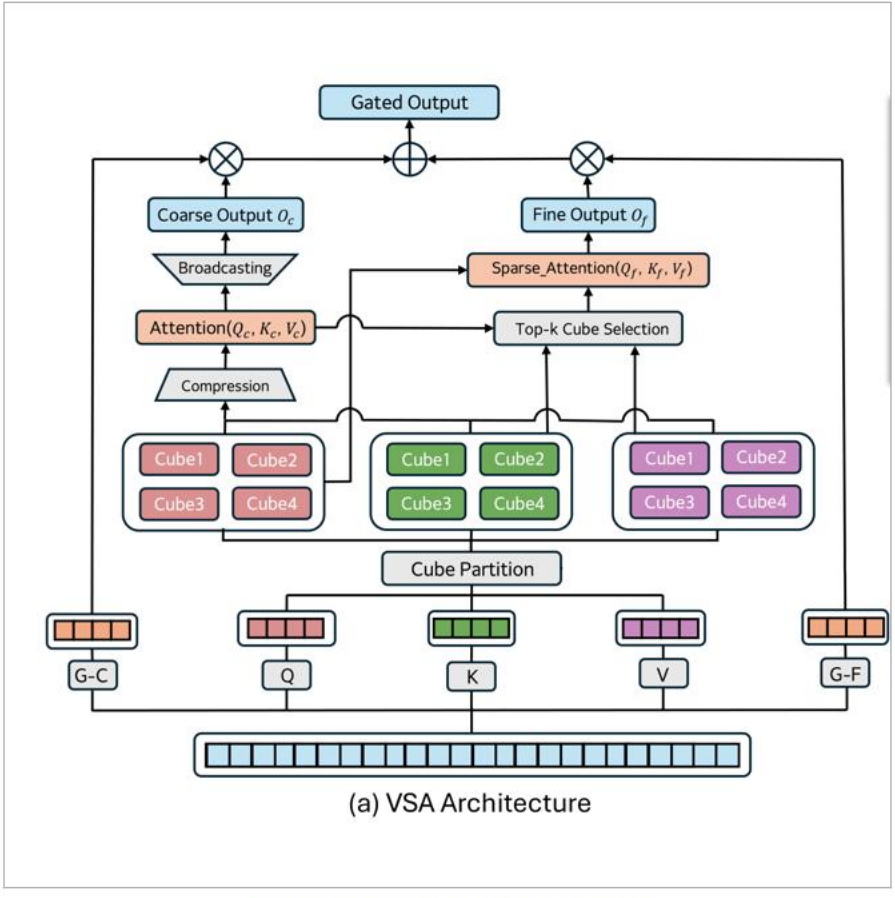
- Coarse attention scans all cubes; Top-K picks where fine attention should work

Bring both views together

- A learned gate mixes global context with selected fine detail

What stays dense

- USP8 still exchanges full-length Q/K/V/O between GPUs



(a) VSA Architecture

$$O = O_c \odot G_c + O_f \odot G_f$$

Global coarse context + selected fine detail · FastH3 adaptation: G_f = 1

Source: Video Sparse Attention, arXiv:2505.13389, Fig. 1 · current H3 Ulysses communication path audited from source

Quantization

QAD trains the deployed NVFP4 path against a frozen BF16 teacher

Why PTQ is not enough

- Four-bit rounding can move the model away from its BF16 output

Same sample, two paths

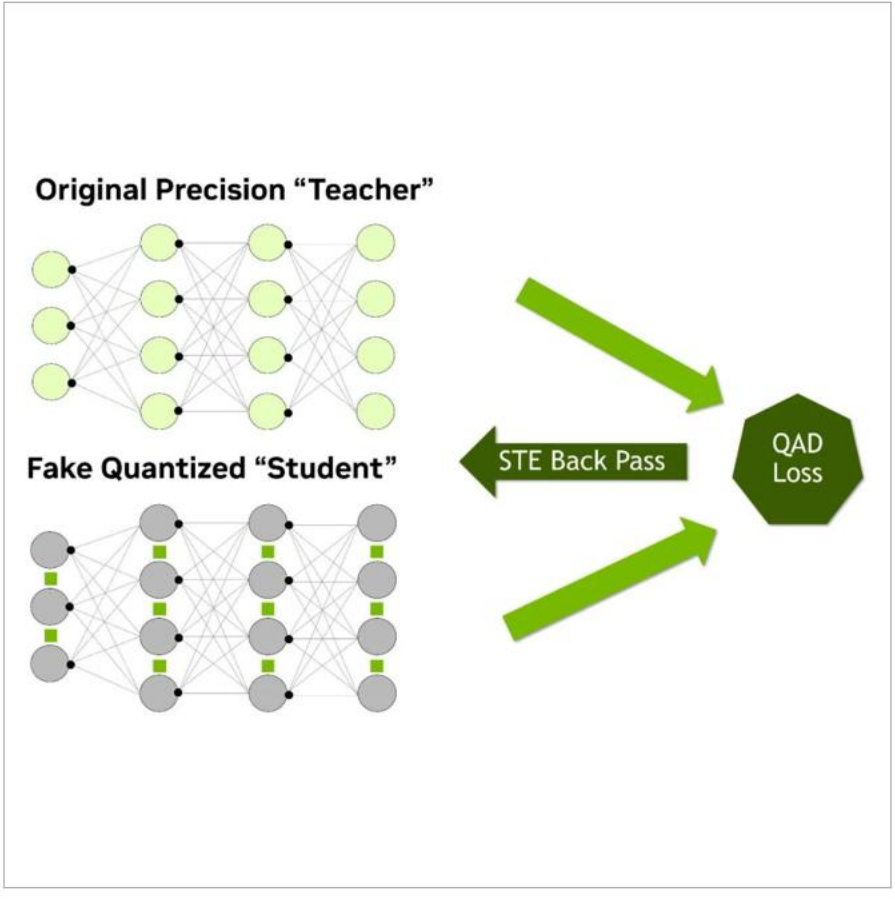
- The teacher stays frozen; the student sees deployment quantization noise

Only the student learns

- Distillation + STE update the student; the teacher disappears at inference

Mixed precision by design

- High-throughput linears use NVFP4; sensitive surrounding math stays BF16



Teacher Guides; NVFP4 Student Learns

Concept: L_QAD = D(y_BF16, y_NVFP4) · the teacher disappears at inference

Sources: NVIDIA Model Optimizer QAD workflow, NVIDIA Technical Blog, Fig. 5 (2025) · FastH3 Omni NVFP4-QAD rev 0087718

Communication

USP8 turns every attention block into four synchronized all-to-all exchanges

Scope	Tensor / Execution	A2A Rounds	Minimum Off-Rank Payload*	Rate at 5.167 s*
One DiT Block	Q / K / V forward + O reverse	4	1.728 GB	61.7 MB/rank/A2A
Base50	50 steps × 50 blocks = 2,500	10,000	4.319 TB	836.0 GB/s
PDD8	8 steps × 50 blocks = 400	1,600	691.1 GB	133.8 GB/s
Boundary	Current VSA / Sage keeps dense BF16 exchange	—	Logical bytes	Not achieved bandwidth

Source-derived @ vLLM-Omni H3 rev 63487a7f · logical one-way off-rank payload lower bound; not measured PCIe/NIC traffic or achieved bandwidth

CUDA Graph

Reuse vLLM’s graph lifecycle around H3’s request-stable compute island

Reuse vLLM’s lifecycle

- Dispatch, capture, cache, replay, and eager fallback stay in the existing framework

Capture only stable H3 work

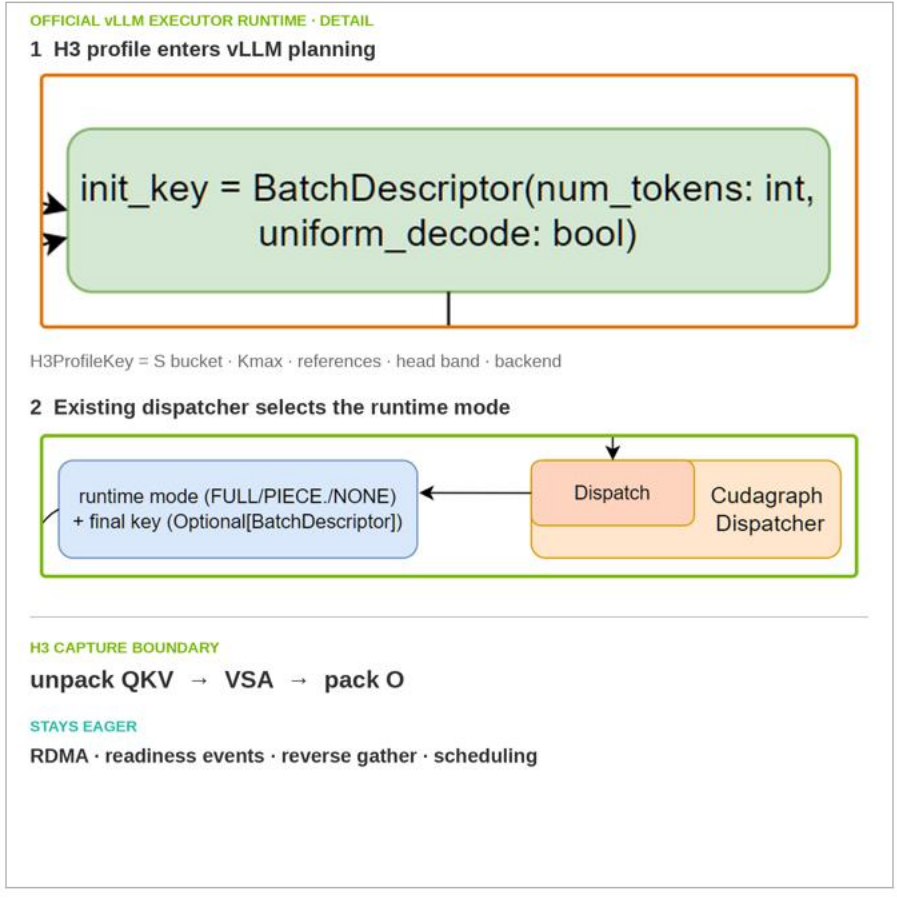
- Fixed buffers replay unpack QKV → VSA → pack O

Leave communication at the boundary

- RDMA, readiness events, reverse gather, and dynamic scheduling remain eager

Measured on one fixed profile

- 8-GPU head pipeline: 13.250 s → 11.994 s (−9.48%)



vLLM Executor Runtime · Detail View

Direct crops of the official planning and dispatch nodes

CUDA Graph Reuse

Video shapes vary across requests, so replay is keyed by a qualified profile

One request repeats the same geometry

- 50 blocks × 8 steps provide 400 replay opportunities per head band; tensor values may change

Match the complete profile

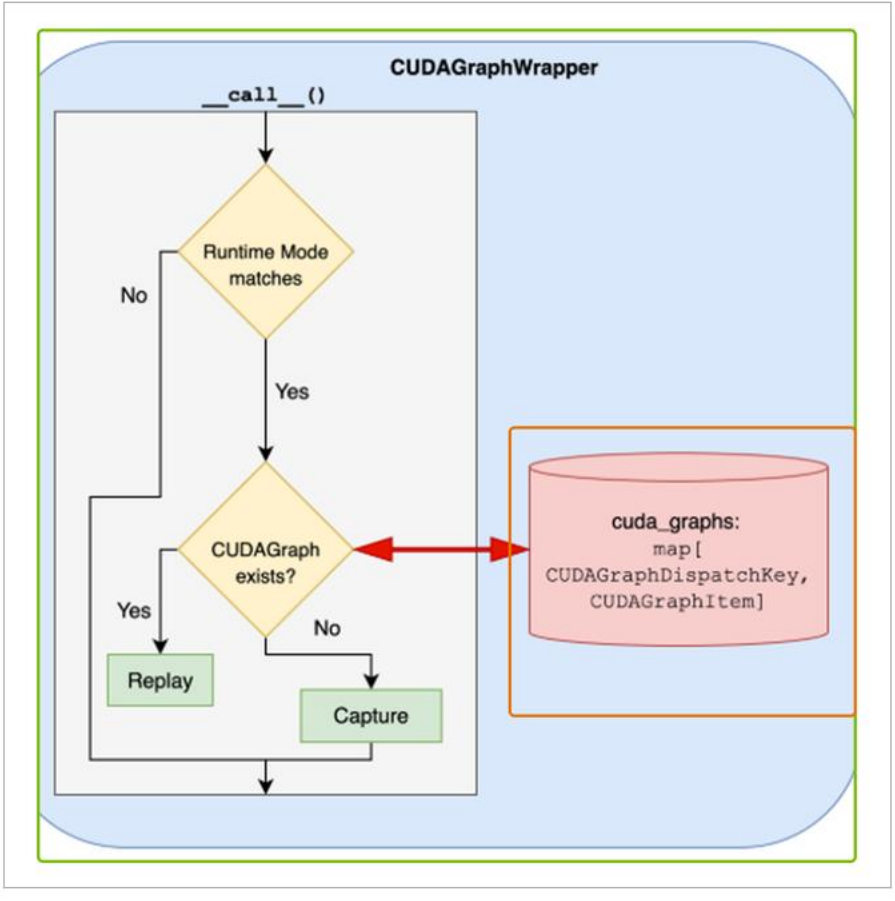
- Packed length S, sparse budget Kmax, reference layout, head band, and backend

On a hit, copy and replay

- Copy current values into stable buffers, then launch the cached graph

On a miss, run eager

- Serve immediately, qualify recurring profiles, then capture; report hit rate, cold cost, and graph memory



vLLM CUDAGraphWrapper · Detail View

H3 extends the dispatch key; the wrapper still owns replay and fallback

USP

Each GPU swaps a token shard for seven full-sequence heads

Start with token shards

- Eight GPUs split 56,960 tokens; each rank keeps 7,120 tokens and all 56 heads

Swap the sharding axis

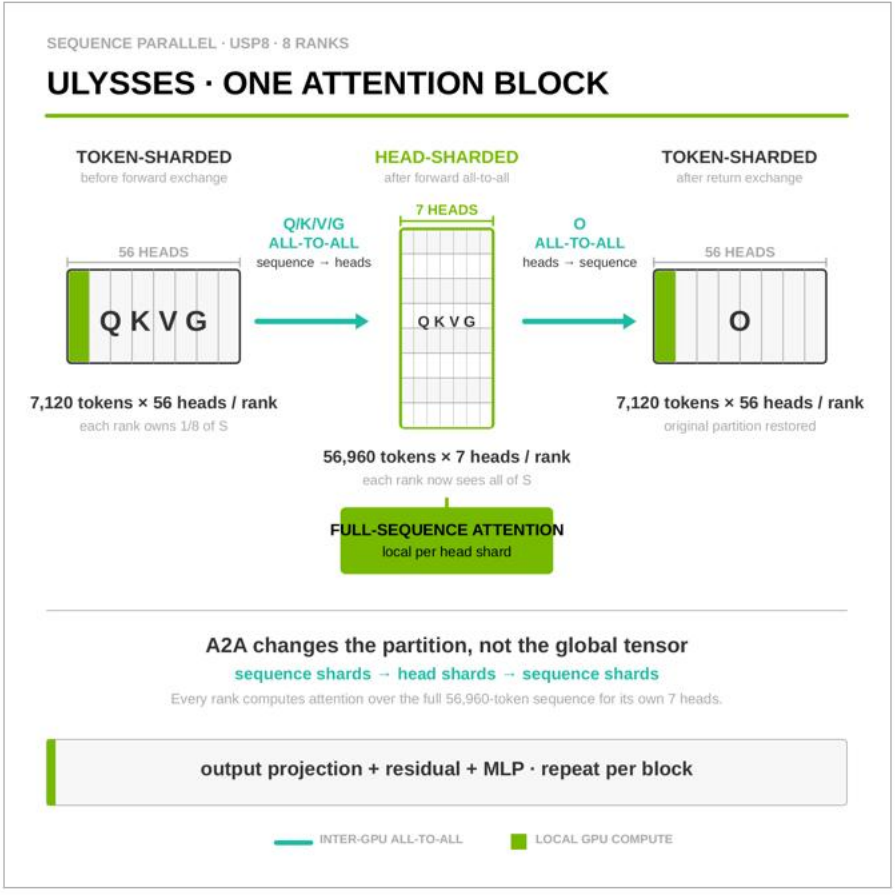
- Q/K/V/G all-to-all changes [S/8, 56] into [S, 7]

Do the expensive part locally

- Each GPU sees the full sequence for its own seven heads

Switch back for the block

- The O all-to-all restores token shards for projection, residual, and MLP



Same Tensor, Different Ownership

7,120 × 56 → 56,960 × 7 → 7,120 × 56 · repeated in every attention block

Adapted from DeepSpeed-Ulysses (Jacobs et al., 2023) · current H3 path: DP1 / TP1 / USP8 / Ring1 · measured 362-frame profile

USP Communication

The exchange is balanced in bytes; the host paths are not

Four PCIe islands

- Each island holds two GPUs and two nearest 400-Gb/s CX-8 ports

Three kinds of peer path

- From one rank: 1 PXB, 2 NODE, and 4 SYS peers

Most peers cross sockets

- Four of seven off-rank peers use the SYS path

Why the baseline is uneven

- IB is disabled, so local NCCL follows these host paths



Eight GPUs · Four PCIe Islands

The math is balanced. The roads are not.

Sources: MiniMaxH3Pipeline and denoise-loop code audit · retained nvidia-smi topo -m · 8 nearest 400-Gb/s CX-8 ports

RDMA

The nearest-NIC path moves the same payload in less than half the measured communication + layout time

Route	Logical Off-Rank Volume	Rank-Max / Block	Effective BW / Rank	Serialized PDD8*
NCCL local	446.6 MB / rank / block 1.429 TB / all ranks / PDD8 request	24.87 ms	17.95 GB/s	9.95 s
Nearest-NIC GDR	446.6 MB / rank / block 1.429 TB / all ranks / PDD8 request	10.86 ms	41.14 GB/s	4.34 s
RDMA delta	Same five-slot bytes	−56.4%	2.29×	−5.61 s
Measured scope	Communication + path-required layout	Exact vs independent all_gather	No attention or GEMM	No overlap *serial × 400; not E2E

Measured 2026-10-25 · 5 warmups + 8 trials × 4 calls · rank-max then mean · *serialized block mean × 400; not end-to-end

RDMA Validation

Four-GPU transport is byte-exact;
eight-GPU H3 validation is still pending

Q/K first

- V waits for an explicit CUDA event

Graph replay

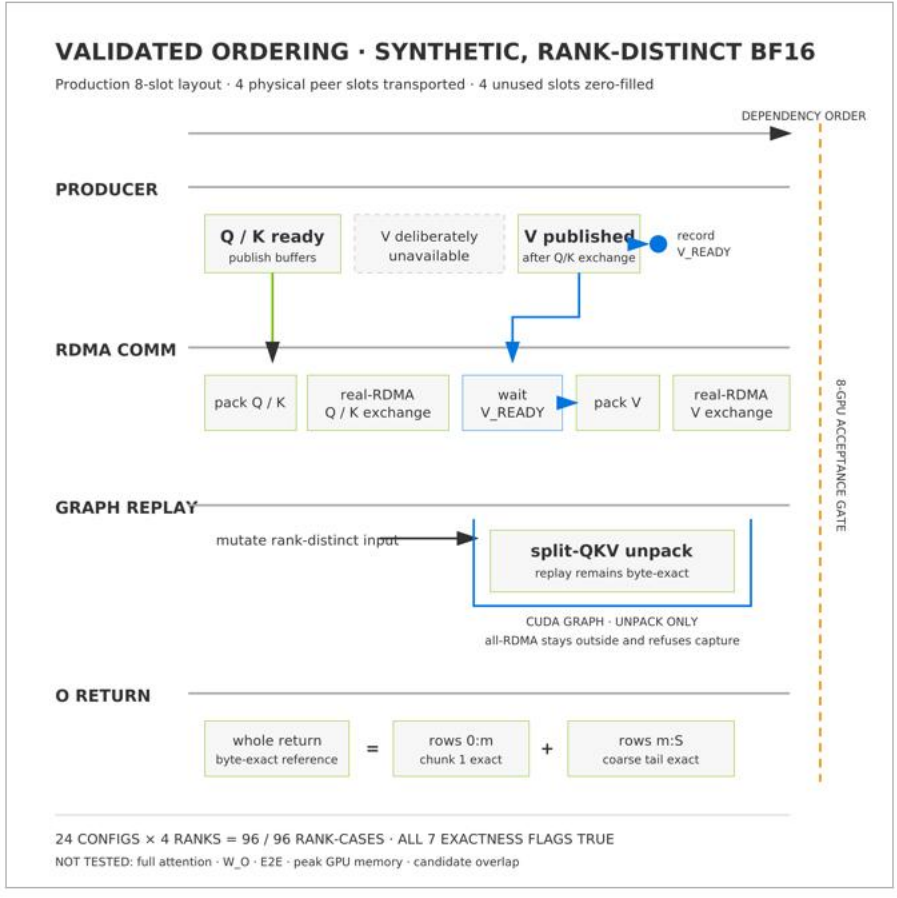
- Fixed-address unpack remains exact after the input changes

Tiled O return

- Whole and row-chunk returns match byte for byte

Eight-GPU acceptance gate

- Full attention + W_O, end-to-end latency, memory, and Nsight overlap



Four-GPU Real RDMA · 96 of 96 Rank Cases Exact

24 configs × 4 ranks · 8-GPU H3 candidate pending

Measured evidence: real-rdma-four-gpu-20261024-v2 · 2026-10-24 · candidate correctness precheck; latency and overlap not measured

Real-Time Demonstrated; Attribution Pending

Only Base50 uses the locked 960×544, 124-frame request; each optimized result uses a different workload

Status	Workload	System	Result / Limit
Measured baseline	960×544 · 124f · 5.167 s	Base50 · BF16 · USP8 · NCCL	247.956 s · RTF 47.992
15 s study	960×544 · 362f · 15.083 s	PDD8 · QAD · VSA · RDMA	14.974 s mean · RTF 0.993
10 s study	960×544 · 243f · 10.125 s	4-step Turbo · MXFP8 · Sage · RDMA	6.958 s · RTF 0.687
Portrait study	576×1024 · 119 new frames · 4.958 s	4-step MXFP8 · Sage · FP4 VAE · RDMA	4.459 s median · RTF 0.899
Required matched test	Lock image, audio, prompt, seed, shape, and request mode	PDD8 → VSA → QAD/Sage → RDMA	At least 3 runs per stage

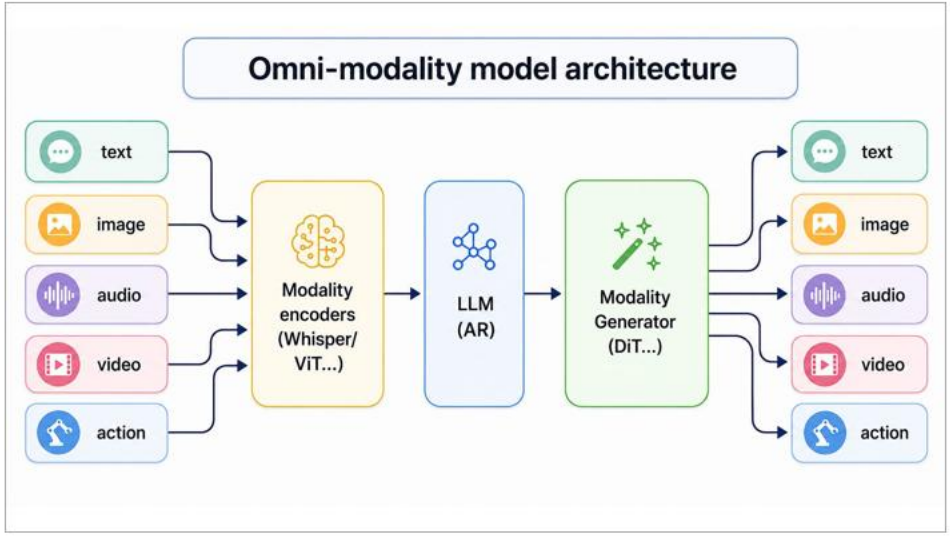
Sources: [A] Base50 r4 · [B] QAD 15 s cohort · [C] paired 10 s study · [D] portrait production · compare only within a row; timing and memory methods differ

02 · ComfyUI Apps With vLLM-Omni



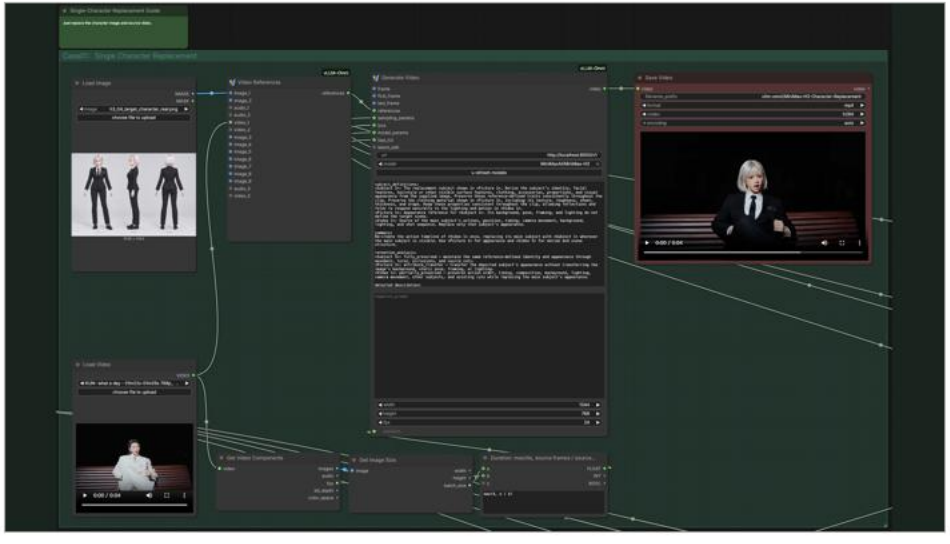
vLLM-Omni Brings Multi-GPU H3 to ComfyUI

Open PR #9194 adds character replacement, continuation-aware references, and chainable LoRA controls



Official vLLM-Omni Architecture

ComfyUI authors the workflow; vLLM-Omni runs scheduling, model execution, and multi-GPU deployment



ComfyUI Workflow · PR #9194

The workflow supports single- and multi-character replacement, source-derived duration, long-video continuation, and LoRA controls

Sources: vLLM-Omni v0.26 public deck, slide 22 · vLLM-Omni PR #9194 (open)

